Universidad de San Carlos de Guatemala

Sistema de Análisis de Rendimiento Académico

INFORME PRACTICA 1

Cristian Ronaldo Sabán Sequén

202303371

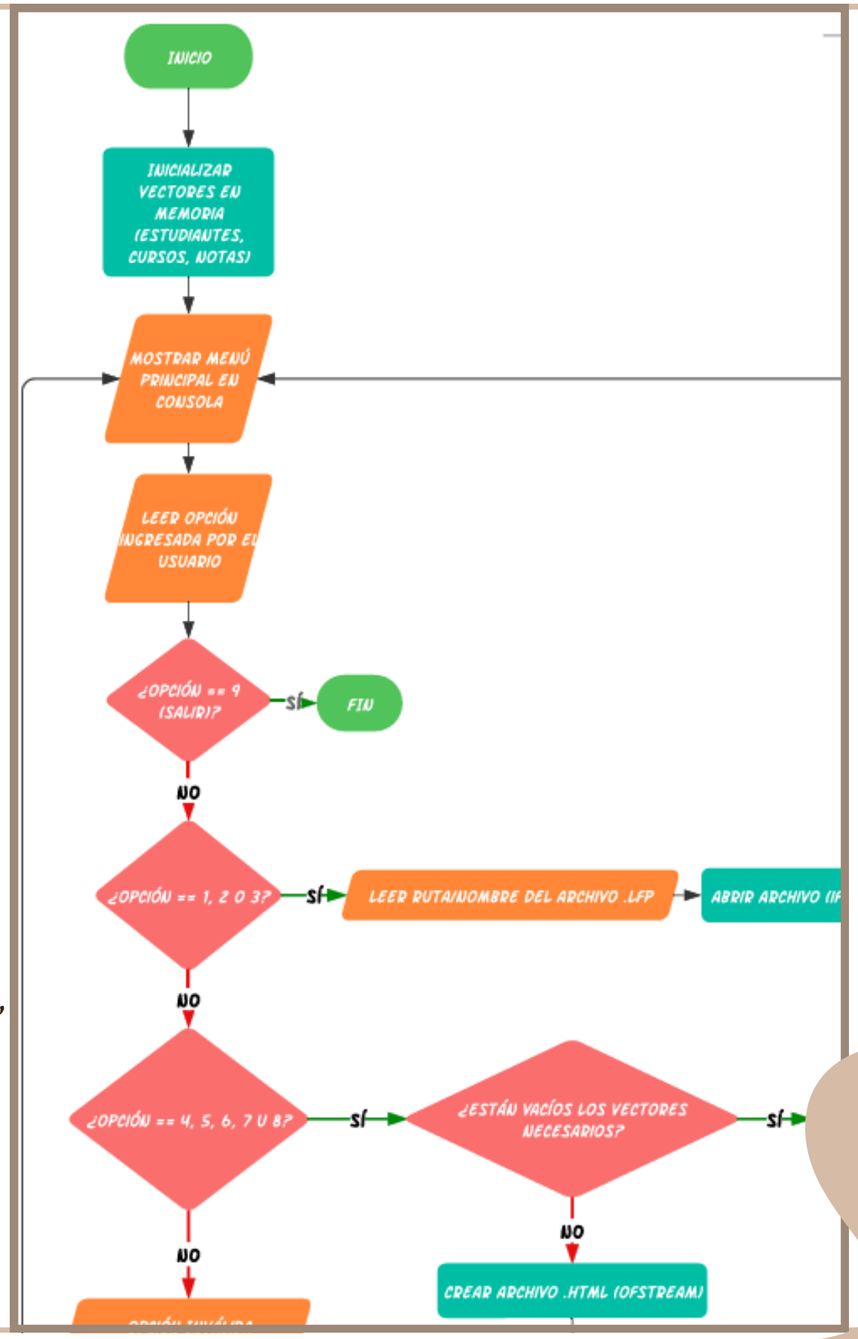
LFP

A

Facultad de Ingeniería - Escuela de Ciencias y Sistemas

IMPLEMENTACIÓN DE LA PRÁCTICA

El desarrollo de esta práctica se llevó a cabo utilizando el lenguaje C++ bajo el paradigma de programación estructurada. El objetivo principal fue construir un sistema de consola capaz de procesar múltiples archivos de texto plano (.lfp) para simular las fases iniciales de lectura y separación léxica de un compilador.
La información extraída (estudiantes, cursos y notas) se modeló utilizando estructuras (struct) y se almacenó en la memoria principal mediante arreglos dinámicos (std::vector). Posteriormente, se implementaron algoritmos para recorrer estas estructuras, calcular métricas estadísticas (promedios, medianas, desviación estándar) y generar dinámicamente cinco reportes en formato HTML puro utilizando la librería <fstream>.



RETOS TÉCNICOS ENCONTRADOS

Ausencia de un método split nativo

A diferencia de lenguajes como Java o Python, C++ estándar no posee una función nativa sencilla para separar cadenas de texto por un delimitador (las comas de los archivos .lfp).

Conversión de Tipos de Datos:

Al leer los archivos de texto, toda la información ingresa como cadenas (string). Representó un reto asegurar que valores numéricos, especialmente las notas con decimales, se convirtieran correctamente para no perder precisión en los cálculos estadísticos.

Ordenamiento Dinámico de Estructuras:

Para los reportes de "Top 10" y "Cursos con Mayor Reprobación", fue necesario ordenar arreglos de objetos complejos basándose en atributos que no existían originalmente en los archivos, sino que eran calculados en tiempo de ejecución (como el promedio o el porcentaje).

```
        }

        double porcentaje = 0.0;
        if (total > 0)
        {
            // Multiplicamos por 100.0 para
            porcentaje = (reprobados * 100.0
        }

        estadisticasCursos.push_back({curso.
    }

    // 2. Ordenamos el vector por porcentaje
    sort(estadisticasCursos.begin(), estadis
         { return a.porcentaje_reprobacion >

    // 3. Imprimimos en el HTML
    archivo << fixed << setprecision(2);

    for (const auto &cursoEst : estadisticas
    {
        archivo << "<tr>"
                << "<td>" << cursoEst.codigo
                << "<td>" << cursoEst.nombre
                << "<td>" << cursoEst.total_
                << "<td>" << cursoEst.aproba
                << "<td>" << cursoEst.reprob
                << "<td>" << cursoEst.porcen
                << "</tr>\n";
```

SOLUCIONES APLICADAS

Para solventar los retos mencionados, se aplicaron las siguientes estrategias:

IMPLEMENTACIÓN DE UN SPLIT PERSONALIZADO:

Se desarrolló una función auxiliar utilizando la clase stringstream y la función getline(). Esto permitió iterar sobre cada línea leída y extraer los tokens separados por comas hacia un vector temporal de cadenas.

CASTEO ESTRICTO:

Se utilizaron las funciones de la librería estándar stoi() (String to Integer) para carnets y semestres, y stod() (String to Double) para asegurar que las calificaciones mantuvieran su valor de punto flotante para operaciones precisas.

USO DE STD::SORT Y FUNCIONES LAMBDA:

En lugar de crear algoritmos de ordenamiento desde cero, se utilizaron estructuras temporales (EstudiantePromedio, CursoReprobacion) y se ordenaron utilizando std::sort() junto con funciones Lambda en línea, lo que permitió definir criterios de ordenamiento descendente de forma muy limpia y eficiente.

RESULTADOS GENERADOS

```
========================================
   SISTEMA DE ANALISIS ACADEMICO
========================================
0. Verificar datos cargados en memoria
1. Cargar archivo de estudiantes
2. Cargar archivo de cursos
3. Cargar archivo de notas
4. Generar Reporte: Estadisticas por Curso
5. Generar Reporte: Rendimiento por Estudiante
6. Generar Reporte: Top 10 Mejores Estudiantes
7. Generar Reporte: Cursos con Mayor Reprobacion
8. Generar Reporte: Analisis por Carrera
9. Salir
Seleccione una opcion:
```

```
Ingrese el nombre/ruta del archivo de estudiantes (ej. estudiantes.lfp): estudiantes.lfp
Exito: Archivo de estudiantes cargado correctamente.
```

EJECUCIÓN DEL MENÚ PRINCIPAL EN CONSOLA

```
Seleccione una opcion: 4
Generando Reporte 1...

============================================================
EXITO: Reporte 1 generado como 'Reporte1_Estadisticas.html'
============================================================
```

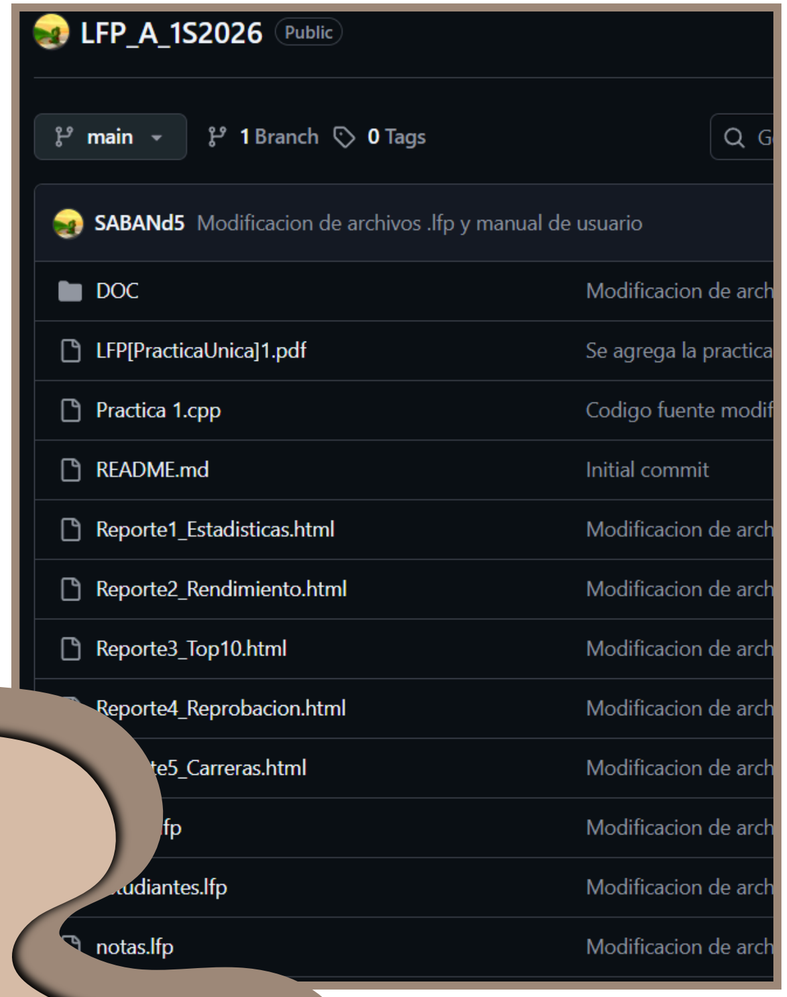
Reporte5_Carreras	02/03/2026 12:25 a. m.	Microsoft Edge HTML Docu...	1 KB
Reporte4_Reprobacion	02/03/2026 12:25 a. m.	Microsoft Edge HTML Docu...	1 KB
Reporte3_Top10	02/03/2026 12:24 a. m.	Microsoft Edge HTML Docu...	1 KB
Reporte2_Rendimiento	02/03/2026 12:24 a. m.	Microsoft Edge HTML Docu...	2 KB
Reporte1_Estadisticas	02/03/2026 07:16 p. m.	Microsoft Edge HTML Document	1 KB

GENERACIÓN DE REPORTES

Reporte 2: Rendimiento por Estudiante

Carnet	Nombre Completo	Carrera	Semestre	Promedio General	Cursos Aprobados	Cursos Reprobados	Creditos Acumulados
202012345	Juan Perez	Sistemas	5	88.67	3	0	15
202013456	Maria Lopez	Industrial	3	78.00	1	0	5
202014567	Carlos Gonzalez	Civil	7	55.00	0	1	0
202100001	Ana Martinez	Sistemas	5	95.00	1	0	5
201900002	Luis Ramirez	Quimica	7	60.00	0	1	0
202201234	Pedro Gomez	Sistemas	4	67.50	1	1	5
202205678	Lucia Fernandez	Civil	2	40.00	0	1	0
202309876	Marcos Ruiz	Industrial	6	65.00	1	0	5
202104567	Elena Morales	Sistemas	8	92.50	2	0	10
202001111	Jorge Castillo	Mecanica	5	30.00	0	1	0
202300099	Sofia Herrera	Quimica	4	55.50	1	1	5

VISUALIZACIÓN DE REPORTES HTML



CONCLUSIONES FINALES

- La manipulación de flujos de entrada y salida (ifstream / ofstream) en C++ demostró ser una herramienta robusta no solo para leer datos estructurados, sino también para generar código de marcado dinámico (HTML) que facilita la visualización de resultados analíticos.

- El uso de la Standard Template Library (STL), específicamente de la colección dinámica vector, permitió una gestión de memoria segura y eficiente, adaptándose dinámicamente a la cantidad de registros en los archivos de entrada sin necesidad de definir tamaños fijos de antemano.

- Simular la fase de separación de datos (tipo analizador léxico) a través de delimitadores reforzó la comprensión de cómo los lenguajes de programación interpretan la información estructurada, sentando las bases para proyectos más complejos en el curso